Date:22-10-26

CO2 ASSESMENT TOOL 1

Analytical problem Solving

Name: B. Pranay Deep Chowdary Course Code: DSA0308

Reg No: 192472177

1. Morphological Analysis of Connected, Connecting, Connection

Aim:

To identify the root words and suffixes, classify them, and convert the words into a common form for document indexing.

Analysis:

The given words are connected, connecting, and connection. All these words are related to the root word connect.

- connected: The suffix -ed indicates past tense. It is inflectional.
- connecting: The suffix-ing, indicates an ongoing action. It is inflectional.
- connection: The suffix -ion changes the verb into a noun. It is derivational.

Morphological Breakdown:

Original Word	Root	Suffix	Type	Normalized Form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Python Program:

```
print ("Name: B.Pranay Deep Chowdary")
```

```
print ("Reg No: 192472177")
words = ["connected", "connecting", "connection"]
print("Word\t\tRoot\tSuffix\tType\t\tNormalized")
for word in words:
    if word.endswith("ed"):
        root = word [:-2]
        suffix = "-ed"
        kind = "Inflectional"
    elif word.endswith("ing"):
        root = word [:-3]
        suffix = "-ing"
        kind = "Inflectional"
    else:
        root = word [:-3]
        suffix = "-ion"
        kind = "Derivational"
    print (word, "\t", root, "\t", suffix, "\t",
          kind, "\t", "connect")
```

Normalized Output:

Word	Root	Suffix	Type	Normalized
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Result: All three words are normalized to the common root connect.

2. Morphological Parsing of Unhappy, Happiness and Happily

Aim:

To identify the prefixes, suffixes, and base words and to classify the transformations for sentiment analysis.

Analysis

The given words are unhappy, happiness, and happily. All three words are related to the base word happy.

- unhappy: The prefix un- gives the opposite meaning of happy.
- happiness: The suffix -ness changes the adjective into a noun.
- happily: The suffix -ly changes the adjective into an adverb.

Morphological Breakdown:

Original Word	Prefix	Base Form	Suffix	Type	Normalized Root
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

Python program:

```
print ("Name: B.Pranay Deep Chowdary")
print ("Reg No: 192472177")
words = ["unhappy", "happiness", "happily"]
print("Word\t\tPrefix\tBase\tSuffix\tType\t\tRoot")
for word in words:
    if word.startswith("un"):
        prefix = "un-"
```

```
        base = word [2:]
        suffix = "None"
    elif word.endswith("ness"):
        prefix = "None"
        base = word [:-4] + "y"
        suffix = "-ness"
    else:
        prefix = "None"
        base = word [:-2] + "y"
        suffix = "-ly"
    print (word, "\t", prefix, "\t", base, "\t", suffix,"\tDerivational\t",
"happy")
```

Output:

Original Word	Prefix	Base Form	Suffix	Type	Normalized Root
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

Result: The program identifies the morphological components and normalizes all three words to happy.

3. Stemming-Based Preprocessing of Played, Player and Playing

Aim

To remove suffixes from words, identify their stems, and normalize them for a search engine.

Analysis

The given words are played, player, and playing. All are related to the word play.

- played: The suffix -ed indicates past tense. It is inflectional.
- player: The suffix -er refers to a person who performs an action. It is derivational.
- playing: The suffix -ing indicates an ongoing action. It is inflectional.

Morphological Breakdown:

Original Word	Stem	Removed Affix	Type	Normalized Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Python program:

```
print ("Name: B.Pranay Deep Chowdary")

print ("Reg No: 192472177")

words = ["played", "player", "playing"]

print("Word\t\tStem\tAffix\tType\t\tNormalized")

for word in words:

    if word.endswith("ed"):
```

```
        stem = word[:-2]
        affix = "-ed"
        kind = "Inflectional"
    elif word.endswith("er"):
        stem = word[:-2]
        affix = "-er"
        kind = "Derivational"
    else:
        stem = word[:-3]
        affix = "-ing"
        kind = "Inflectional"
    print(word, "\t", stem, "\t", affix, "\t",
          kind, "\t", "play")
```

Output:

Original Word	Stem	Removed Affix	Type	Normalized Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Result: The program extracts the common stem play from all three words.

4. Finite-State Morphological Parsing of Writes, Writing and Written

Aim

To design a finite-state parser that recognizes regular and irregular verb forms and converts them to a common root.

Analysis

The given words are writes, writing, and written. All three words are forms of the verb write.

- writes: The suffix -s indicates the third-person singular present tense.
- writing: The suffix -ing indicates an ongoing action. The final e in write is removed before adding -ing.
- written: This is an irregular past participle of write.

Current State	Condition	Next State	Meaning
q0	Start	q1	Begin parsing
q1	Word is writes	q2	Identify present-tense form
q1	Word is writing	q3	Identify present-participle form
q1	Word is written	q4	Identify irregular form
q2, q3, q4	Root identified	qf	Accept the word

Morphological Breakdown

Word	Breakdown	Type	State Path	Normalized Form
writes	write + s	Regular inflection	q0 → q1 → q2 → qf	write
writing	writ(e) + ing	Regular inflection	q0 → q1 → q3 → qf	write
written	Irregular form	Irregular inflection	q0 → q1 → q4 → qf	write

Python program:

```
print("Name: B.Pranay Deep Chowdary")
print("Reg No: 192472177")
words = ["writes", "writing", "written"]
print ("Word\t\tState Path\t\t\tBreakdown\tType\t\tNormalized")
for word in words:
    if word == "writes":
        path = "q0 -> q1 -> q2 -> qf"
        breakdown = "write + s"
        kind = "Regular"
    elif word == "writing":
        path = "q0 -> q1 -> q3 -> qf"
        breakdown = "writ(e) + ing"
        kind = "Regular"
    else:
```

```
        path = "q0 -> q1 -> q4 -> qf"
        breakdown = "Irregular form"
        kind = "Irregular"
    print (word, "\t", path, "\t", breakdown,
          "\t", kind, "\t", "write")
```

Output:

Word	Breakdown	Type	State Path	Normalized Form
writes	write + s	Regular inflection	q0 → q1 → q2 → qf	write
writing	writ(e) + ing	Regular inflection	q0 → q1 → q3 → qf	write
written	Irregular form	Irregular inflection	q0 → q1 → q4 → qf	write

Result: The finite-state parser recognizes all three forms and normalizes them to write.

5. Porter Stemmer-Based Preprocessing of Relational, Relation and Relate

Aim

To use the Porter Stemming algorithm to remove suffixes and obtain common stems for document retrieval.

Analysis

The given words are relational, relation, and relate. The Porter Stemmer removes suffixes according to its predefined rules.

- relational: The suffix -ational is reduced through the Porter stemming rules, and the word eventually becomes relat.
- relation: The suffix is removed according to the algorithm's rules, producing relat.
- relate: The final e is removed, producing relat.

Stemming Table:

Original Word	Stemming Process	Intermediate Form	Final Stem
relational	Reduce -ational and remove final e	relate → relat	relat
relation	Apply suffix-removal rules	relat	relat
relate	Remove final e	relat	relat

Python program:

```
from nltk.stem import PorterStemmer
print ("Name: B.Pranay Deep Chowdary")
print ("Reg No: 192472177")
stemmer = PorterStemmer()
words = ["relational", "relation", "relate"]
print ("Word\t\tFinal Stem")
for word in words:
    stem = stemmer.stem(word)
    print (word, "\t", stem)
```

Output:

Original Word	Final Stem
relational	relat
relation	relat
relate	relat

Result: The Porter Stemmer reduces all three words to relat, which can be used for document indexing and information retrieval.